

Art & Animation Specification — *Pose: Caribbean Dominoes*

For: the artist producing tiles, board, destruction and motion for *Pose* **From:** INVOVIBE TECH LTD **Engine:** Unity 6 LTS (URP) · **DCC:** Blender or Maya, your choice **Status:** v1 — double-six scope **Read alongside:** [PROJECT_BRIEF.md](#) for what the product is

0. Read this part first

This is a mobile game, not a film. Three things follow from that, and they override any other instruction in this document:

1. **It must hold 60fps on a Pixel 5a.** That is the published technical bar ([PROJECT_BRIEF.md](#)). Polygon and texture budgets in §7 are hard ceilings, not targets to approach.
2. **The player looks at a 6-inch screen.** A domino in hand is roughly 100–140 screen pixels tall. Detail that doesn't survive at that size is wasted work. Test early and often at true size (§3.4).
3. **Nothing you make decides anything.** All of it is presentation. The rules, the deal, and the outcome are computed elsewhere. This has one concrete consequence you must design around, in §5.1.

You are not expected to know Unity. Everything here is specified so you can work entirely in Blender or Maya and hand over files. Terms that may be unfamiliar are defined in §11.

1. The vision

Pose is built around Caribbean dominoes, where **the slam is the point**. Players don't place a tile, they *land* it. The table takes it. That physicality is the product's signature, and it is the thing you are being asked to build.

The feel we're chasing, in order:

Weight. Every tile has mass. It doesn't glide into position — it drops, hits, and settles. The player should feel the tile land through the screen.

Escalation. The board is not static scenery. It accumulates damage across a round. Early plays leave hairline marks. Big plays crack it properly. The round-winning tile *destroys it* — chunks flying, dust, the works — and then it reforms for the next round.

Restraint between the peaks. The destruction only reads as a payoff if most moves are quiet. A standard placement is a modest thump. The spectacle is rationed.

The reference for overall polish level is Ludo Club: crisp, tactile, sound and animation locked frame-to-frame, particles on every meaningful beat.

Art direction (colour, material, setting) is not yet fixed. §10 lists the decisions needed from Giselle before you begin final art. Start with §9 Phase 0 style frames — that conversation is much easier with pictures.

2. What you are delivering

#	Deliverable	Type	§
1	SM_Tile_Blank — one domino mesh	Mesh	3.1
2	Pip atlas — 7 half-face designs	Texture	3.2
3	Tile back + edge materials	Texture	3.3
4	SM_Board_Intact	Mesh	4.1
5	Crack decal library — 4 variants	Texture	4.2
6	SM_Board_Fractured — 40–60 chunks	Mesh	4.3
7	VFX flipbooks — dust, debris, shockwave, flash	Texture	6
8	Motion reference — 4 sequences, video + timing sheets	Video + doc	5

Eight items. That is the whole job. Note what is **not** on the list: 28 tile meshes, baked animation clips, particle systems, or anything Unity-specific. Those are either unnecessary (§3) or built on our side from your reference (§5).

3. The tile

3.1 Mesh — SM_Tile_Blank

You model **one** blank domino. All 28 tiles in the set are this mesh with different pips drawn onto it at runtime.

Property	Spec
Dimensions	50 × 25 × 9 mm (real-world domino)
Triangles	≤ 300
Bevel	Yes — small, ~0.6mm, 2 segments. This is what catches light and sells the material
Pivot / origin	Exact centre of the tile, all three axes
UV	Face → atlas region (§3.2). Back and edges → their own regions. No overlapping UVs
Material slots	One. Face, back and edge all read from a single atlas
Centre divider	Model it as geometry (a shallow groove), do not rely on texture alone

Model at true real-world size. Do not scale up for physics reasons — we handle that at the engine end. Your file should be physically correct.

A single material slot matters more than it sounds: every extra material is an extra draw call, and there can be 28 tiles plus a board on screen at once.

3.2 The pip atlas — T_TilePips_BC

This is the highest-value asset in the game and the one that makes the skin business work.

A domino face is two square halves. With double-six, each half shows 0–6 pips — **seven designs total**. You draw those seven once; the engine composes [5|2] by sampling cell 5 and cell 2 onto the two halves of the face.

T_TilePips_BC – 1024 × 512, 4×2 grid, 256px cells

0 blank	1 •	2 • •	3 • • •
4 • • • •	5 • • • • •	6 • • • • • •	spare

Cell N contains the pip layout for N pips, centred, on the tile's face colour.
Bleed 4px of face colour past each cell edge to prevent mip bleeding.

The layouts themselves are the standard dice arrangements — there is nothing to invent. **The entire craft here is spacing, weight and rhythm.** Pips that are individually gorgeous but read as a grey smudge at 120px have failed. Pips with slightly too much air read as cheap. This is the asset to spend your time on.

The spare eighth cell is reserved — leave it as flat face colour for now.

3.3 Back and edges

Asset	Notes
Tile back	Occupies its own UV region. The back is seen constantly — every opponent hand, the whole shuffle. Give it a treatment: pattern, logo plate, subtle sheen
Tile edge	Usually a plain material band. Include it in the atlas layout

3.4 The legibility test — do this before you finish

Before detailing, render your atlas onto the tile mesh, screenshot at **128 pixels tall**, and look at it on a phone. Can you read the pip count instantly, without counting? Can you tell [6|5] from [6|6] at a glance?

If not, adjust and repeat. Do this at 20% completion, not at 90%.

3.5 Why you are not making 28 tiles

Because a premium tile skin — the thing subscribers pay for ([PROJECT_BRIEF.md](#)) — then costs **seven half-faces, a back and an edge**. Under a day of work per skin. If tiles were 28 individual assets, each skin would be a month and the business model wouldn't function. The atlas structure *is* the monetisation strategy.

Forward compatibility: two rulesets in the catalogue use larger sets (double-9, double-12). If those ship, the atlas extends to 13 cells on the same grid. Nothing you build now gets thrown away — do not design for it, just don't do anything that would prevent it.

4. The board

4.1 [SM_Board_Intact](#)

Property	Spec
Play surface	900 × 900 mm (a card table), 40mm slab thickness
Triangles	≤ 2,000
Pivot	Centre of the play surface, on the top face (Y = 0 at the surface tiles rest on)
Textures	2048 ² max — BaseColor, Normal, Metallic+Smoothness
Edges	Slight bevel or moulding. The board silhouette is on screen the entire match

That pivot rule matters: we position tiles relative to the board surface, so "surface = zero" removes a whole class of alignment bugs.

4.2 Crack decals — [T_BoardCrack_01 ... _04](#)

Damage does **not** come from swapping full-board textures. It comes from stamping crack decals at the actual impact point, so a slam on the left edge cracks the left edge.

Deliver **four variants**, 1024² each, RGBA channel-packed:

Channel	Contents
R	Crack line mask — white where the crack is
G	Depth darkening / ambient occlusion in the crack
B	Reveal order — 0 at the crack's origin point, ramping to 1 at its furthest extent
A	Overall coverage / alpha falloff

The B channel is what makes cracks *grow* rather than pop in. We drive a single 0→1 value and the crack propagates outward from its origin. Paint it as a rough radial gradient following the crack's spread — it doesn't need to be precise.

Each variant should originate near the centre of its texture and radiate outward, roughly 600mm across at board scale. Vary the character: one fine and spidery, one forked and aggressive, one short and blunt, one long and branching.

4.3 SM_Board_Fractured — the destruction mesh

This is the payoff asset and the most interesting piece of the job.

Property	Spec
Chunks	40–60 . Fewer reads as cheap, more costs framerate
Total triangles	≤ 15,000 across all chunks
Assembled silhouette	Must match SM_Board_Intact exactly — we swap one for the other on a single frame and the cut must be invisible
Chunk pivots	Each chunk's origin at its own centre of mass — non-negotiable, chunks rotate about their pivots
Interior faces	Second material — rough, unfinished, broken-inside look. The inside must not look like the polished top
Hierarchy	All chunks parented under one empty/group named SM_Board_Fractured
Chunk naming	SM_Board_Chunk_01 ... SM_Board_Chunk_NN , zero-padded

Property	Spec
Chunk size	Vary it. Some large slabs, many mid, a scatter of small. Uniform chunks look procedural

Producing the fracture:

- **Blender** — the Cell Fracture add-on (Edit → Preferences → Add-ons → "Object: Cell Fracture"). Use a noise or weighted point source biased toward the board centre so the break radiates from impact. Then: set each chunk's origin via Object → Set Origin → **Origin to Center of Mass (Volume)**.
- **Maya** — Effects → Shatter → Solid Shatter, or the Bullet plugin's fracture tools. Then Modify → **Center Pivot** on every chunk.

Either is fine. If you have a preference, Blender's Cell Fracture gives more control over the point distribution for this kind of radial break.

Assign the interior material inside the fracture step, before you clean up — separating inner from outer faces afterwards by hand across 50 chunks is miserable.

5. Motion

5.1 The one rule that constrains everything

Your animations do not decide outcomes. They visualise decisions already made.

The tile shuffle is the clearest case. The deal is computed from a seed issued by our server *before any animation plays* — this is a cheat-prevention measure and it is not negotiable ([ARCHITECTURE.md](#)).

So the shuffle can look as chaotic as you like in the middle, but it **must resolve to a fixed, predetermined arrangement**. Design it as: chaos → order, with the ending locked. Think of a magician's shuffle where the result was never in doubt.

The same applies to the board shatter: it fires *because* a round was already won. It never causes anything.

5.2 What you deliver, and what you don't

You deliver **motion reference**, not baked animation clips:

- A **playblast video** at 60fps for each sequence
- A **timing sheet** — the beat breakdown, in frames, with the easing on each beat

You do **not** deliver FBX animation. The final motion is coded, because it has to adapt to 2, 3 and 4 players and to variable tile counts — a baked 28-tile shuffle clip would be unusable. Your reference is what we build against, and it is the thing that determines whether the game feels good.

Name your easing in DOTween's vocabulary so the timing sheet maps straight to code: `easeOutBack`, `easeInQuad`, `easeOutElastic`, `easeInOutCubic`, `easeOutBounce`, `linear`. If you specify "punchy overshoot on the settle," we have to guess. If you specify `easeOutBack`, `overshoot 1.7`, we don't.

5.3 Sequence A — Standard placement (~283ms)

The workhorse. Plays on every ordinary move, so it must never become tiresome.

Beat	Frames @60	ms	Easing
Lift / anticipation	5	83	<code>easeOutQuad</code>
Strike — arc down	4	67	<code>easeInQuad</code>
Impact frame	—	—	sound + haptic fire here
Bounce and settle	7	117	<code>easeOutBack</code>
Total	16	~283	

Modest. A thump, not an event.

5.4 Sequence B — Hero slam (~600ms)

Fires on the plays that matter: a capicú, killing a double, the round-winner. This is the Caribbean table slam.

Beat	Frames @60	ms	Notes
Windup — high raise, brief hold	13	217	The hold is what creates anticipation
Strike	5	83	<code>easeInCubic</code> — accelerate hard
Impact + hitstop	4	67	Everything freezes. See §11
Shockwave + dust spawn	—	—	on the impact frame
Tile overshoot and settle	14	233	<code>easeOutBack</code> , overshoot ~1.7
Camera shake decay	11	183	overlaps the settle
Total	~36	~600	

The hitstop is the single highest-value frame in this game. Four frames of total freeze at impact is what converts a fast animation into a *hit*. Storyboard it deliberately.

5.5 Sequence C — Shuffle (~2.6s)

Beat	Frames @60	ms	Notes
Gather from stacks	24	400	tiles lift, face-down
Swirl	96	1600	must loop seamlessly — matchmaking may hold here
Settle into stacks	36	600	resolves to the fixed arrangement (§5.1)
Total	156	~2600	

Make the swirl section loopable. Sometimes we need to hold this while waiting on a network response, and a visible seam every 1.6 seconds is worse than no shuffle at all.

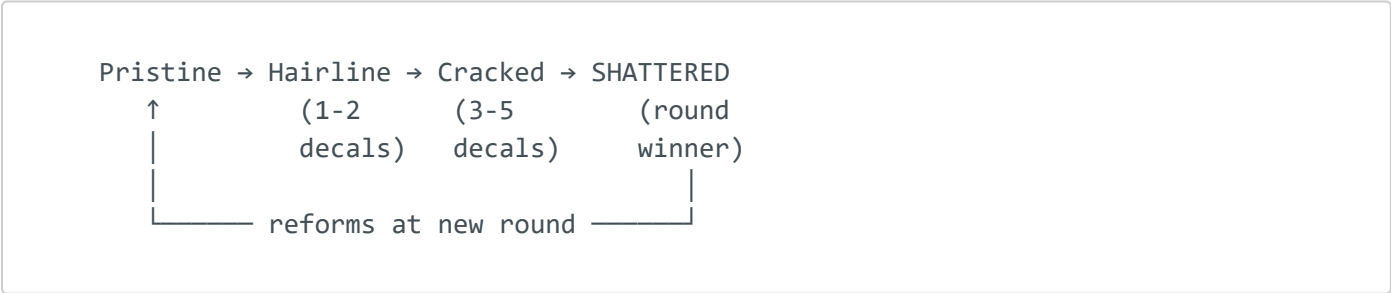
At 28 tiles, a genuine physics-driven jostle is affordable on mobile. Worth prototyping both a physics version and a hand-choreographed one — the hand-animated version usually reads better, but physics gives you happy accidents to steal.

5.6 Sequence D — Shatter and reform

Beat	Frames @60	ms	Notes
Impact + hitstop	5	83	longer freeze than the hero slam
Chunk launch	0–12	0–200	radial from impact, upward bias
Tumble and settle	54	900	
Dust lingers	84	1400	overlaps everything above
Reform	48	800	chunks return to intact position
Total	~190	~3.2s	

The board must come back. Rounds continue. Show me in reference how you want the reform to read — chunks flying back in reverse, or a dissolve-and-materialise, or something better. This is the one beat with no obvious answer and I'd rather have your proposal than my guess.

Why the shatter only fires on a round-ending tile: dominoes sit *on* the board. If it shatters mid-round the layout goes with it. Tying full destruction to the round-winner means the board is free to be destroyed — nothing has to keep playing on it — and the reform covers the round transition. The escalation ladder:



6. VFX flipbooks

Sprite sheets, not particle systems — we assemble the particle systems in Unity from your sheets.

Asset	Grid	Size	Frames	Notes
T_VFX_Dust	8×8	2048 ²	64	Soft dust puff, impact and shatter
T_VFX_Debris	8×8	2048 ²	64	Splinters and grit

Asset	Grid	Size	Frames	Notes
T_VFX_Shockwave	4×4	1024 ²	16	Expanding ground ring
T_VFX_Flash	4×4	1024 ²	16	Impact flash, short and bright

Greyscale on black where possible — we tint in-engine, so one sheet serves every board theme. Author at 30fps playback. Sequence left-to-right, top-to-bottom.

7. Technical specification

The section that prevents a week of rework. None of these are stylistic.

7.1 Universal

Item	Spec
Mesh format	.fbx
Texture format	.png, 16-bit not required
Up axis	Y-up, -Z forward
Transforms	Frozen/applied — must import as position 0, rotation 0, scale 1
Scale	Real-world. Tile 50×25×9mm, board 900×900×40mm
Texture dimensions	Power-of-two, always
Normal maps	Tangent-space, OpenGL convention (+Y up / green up)
Colour space	BaseColor in sRGB. Normal, Mask, and packed data maps in linear
Naming	ASCII only. No spaces, no accents, no #, &, (
Embedded media	Off. Textures ship as separate files

7.2 Texture set per material

Map	Suffix	Packing
Base colour	_BC	RGB colour, A = alpha if needed
Normal	_N	Tangent-space
Mask	_MS	R = metallic, A = smoothness

Three maps per material, maximum. Ambient occlusion bakes into base colour.

7.3 Blender export

Set up: Scene Properties → Units → Metric, Unit Scale 1.0, Length: Millimeters
 Before: Ctrl+A → All Transforms (on every object)
 Delete unused modifiers, or apply them

File → Export → FBX (.fbx)
 Path Mode Copy, Embed Textures OFF
 Limit to Selected Objects
 Object Types Mesh (+ Empty for the fracture group)
 Transform:
 Scale 1.00
 Apply Scalings .. FBX All
 Forward -Z Forward
 Up Y Up
 Apply Unit ON
 Geometry:
 Smoothing Face
 Tangent Space ... ON
 Bake Animation OFF

7.4 Maya export

Set up: Preferences → Settings → Up axis: Y, Linear: centimeter
 Before: Modify → Freeze Transformations
 Edit → Delete by Type → History
 Check for non-uniform scale – remove it

File → Export Selection → FBX
 Geometry:
 Smoothing Groups ON
 Tangents and Binormals ... ON
 Triangulate ON
 Embed Media OFF
 Advanced → Axis Conversion → Up Axis: Y
 Animation OFF

7.5 Naming convention

Prefix	Meaning	Example
SM_	Static mesh	SM_Board_Chunk_07
T_	Texture	T_TilePips_BC
M_	Material (we create these)	M_Tile_Classic

Suffix textures with their map type: `_BC`, `_N`, `_MS`.

7.6 Budgets — hard ceilings

Asset	Triangles	Textures
Tile	300	1024×512 atlas
Board intact	2,000	2048 ²
Board fractured (all chunks)	15,000	2048 ² + interior
Crack decal	—	1024 ² × 4
VFX sheet	—	2048 ² × 2, 1024 ² × 2

8. Delivery

8.1 Folder structure

```
delivery/
├── meshes/
│   ├── SM_Tile_Blank.fbx
│   ├── SM_Board_Intact.fbx
│   └── SM_Board_Fractured.fbx
├── textures/
│   ├── T_TilePips_BC.png
│   ├── T_Board_BC.png  _N  _MS
│   ├── T_BoardCrack_01..04.png
│   └── T_VFX_Dust.png  ...
├── reference/
│   ├── A_standard_placement.mp4
│   ├── B_hero_slam.mp4
│   └── C_shuffle.mp4
```

```
|   └─ D_shatter_reform.mp4
|   └─ timing/
|       └─ timing_sheets.md
```

8.2 Source files stay out of the repo

`.blend`, `.ma` / `.mb`, `.psd`, `.spp` — **do not** put these in the Unity project. They belong in a separate `art-source/` location or shared drive. Only exports cross over.

The Unity project uses Git LFS for binary assets, and source files would bloat it badly. It also uses a system called Addressables for loading art — which means **nothing may go in a folder named `Resources`**. If you're ever placing files into the Unity project directly, that's the one hard prohibition.

8.3 Working method

Send work-in-progress early and often. A grey-boxed tile at correct scale with a correct pivot is worth more to us on week one than a finished tile on week four — placeholder assets unblock the entire game-side build. See §9.

9. Order of work

Sequenced so the game is never blocked waiting on art.

Phase	Deliverable	Purpose
0	Style frames — 2D paintings, no 3D	Settle art direction cheaply, before any modelling
1	Grey-box tile + board — untextured, correct scale, pivot, naming	Unblocks all game-side work immediately. Do this in week one
2	Hero tile — mesh, pip atlas, back, edge	The most-viewed asset in the game
3	Board intact + crack decals	Completes the standard match view

Phase	Deliverable	Purpose
4	Fractured board + VFX sheets	The payoff
5	Motion reference — all four sequences	Can run parallel with 3–4
6	Additional skins — texture-only	Ongoing. This is the revenue engine

Phase 1 is the one people skip. Please don't — an untextured box of the right size with the right pivot is genuinely the highest-leverage thing you can deliver, because it lets the entire rest of the game get built while you make the real art.

10. Open questions — for Giselle, before Phase 2

These are direction calls, not art problems. Phase 0 style frames should be used to answer them.

1. **Tile material** — ivory/bone, polished acrylic, aged wood, something else?
2. **Pip treatment** — recessed and painted, inlaid, printed flat? Colour: classic black, or per-suit colour coding for readability?
3. **Board setting** — a domino table, a bar top, a yard table, an abstract stage?
4. **Board material** — this determines what "breaking" means. Wood splinters. Stone cracks and crumbles. Painted metal buckles. Pick one; it drives §4.3 and §6 entirely.
5. **Camera** — fixed top-down, or a slight angle? Affects how much of the tile's side and bevel is ever seen.
6. **Skin roadmap** — how many themes at launch, and are they regional (Jamaica, Cuba, Trinidad, Puerto Rico)?

Question 4 is the blocker for the destruction work. The others can trail.

11. Glossary

Terms used above that come from games rather than film or DCC work.

Term	Meaning
Atlas	One texture holding many separate images. Cheaper than many textures — the GPU binds it once
Draw call	One instruction to the GPU. Each material on screen costs at least one. The main mobile performance limit
Pivot / origin	The point an object rotates and scales around. Wrong pivots are the most common handoff bug
Tri budget	Triangle ceiling. Quads count as two
Tangent-space normal map	The purple-looking map encoding surface detail. Must be OpenGL convention here
Channel packing	Storing unrelated greyscale data in R, G, B and A of one texture to save memory
Decal	A texture stamped onto a surface at runtime at an arbitrary position — how cracks appear where the tile actually hit
Flipbook	Animation as a grid of frames in one texture, played by scrolling UVs
Rigidbody	An object under physics simulation. Each fractured chunk becomes one
Hitstop	Freezing all motion for a few frames at the moment of impact. Cheap, and the single strongest tool for making a hit feel heavy
Easing	The acceleration curve of a motion. <code>easeOutBack</code> overshoots then settles; <code>easeInQuad</code> accelerates in
Addressables	Unity's system for loading art on demand — how skins download without a new app version
Prefab	A reusable assembled game object. We build these from your meshes; you don't need to
URP	Universal Render Pipeline — Unity's mobile-oriented renderer. Determines the three-map material setup in §7.2

Questions on anything in this document should go to Giselle. Specifications here that conflict with `ARCHITECTURE.md` are wrong — that document wins.

